

# Computational Introduction to Robotics Project 1

## Finite State Machine

Thanadetch (Detch) Mateedunsatits

[dmateedunsatits@olin.edu](mailto:dmateedunsatits@olin.edu)

Luke Wei

[lwei2@olin.edu](mailto:lwei2@olin.edu)

September 21, 2026

### 1 Introduction

The goal of this first project is to gain familiarity with ROS and practice with the Neatos. We will be implementing a few behaviors in this system and creating a finite state machine diagram to visualize and keep it all together.

### 2 System Design — FSM

We implemented a simple finite state machine based on four fundamental states that the Neato could be in, which represent specific types of behavior it can achieve. It will always begin by attempting to follow a wall, and then, after a timer of 60 seconds, it will attempt to drive in a square. It can always be disturbed by two events: collision avoidance and person-following, which activate when the LIDAR scanner detects that it is too close or when it believes there's a person to follow. The way the controller knows about all of these is through keeping two custom topics `/neato_state` to track the Neato's state and `/fsm_events` to track data that would influence the state the Neato is in, which are the person-following and collision-avoiding behaviors.

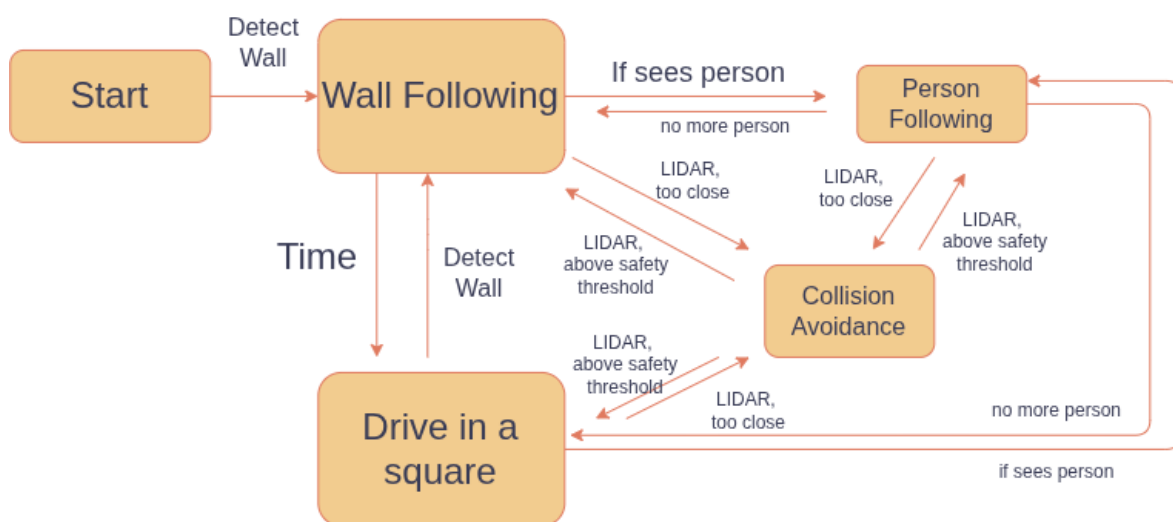


Figure 1 displays the possible states of the Neato and the transitions between them.

## 2.1 Wall Following

Wall-following is the starting behavior of the finite state machine and the Neato will always move in the direction parallel to a wall. The Neato will begin driving forward until it scans a wall nearby, at which point it will home in and begin to move in the direction of a wall. The logic here is that the lidar will send out two scans, and the reading between **r\_perp** and **r\_fwd** will give us the information we need. **r\_perp** is the scan perpendicular to the Neato, and **r\_fwd** is the direct scan; we draw a triangle between these two lines in order to define a variable **alpha**, which will represent the angle between the robot's heading and the wall. When **alpha**  $\approx 0$ , then we can the movement of the Neato is roughly parallel to the wall. It can be visualized from the figure below that when **alpha**  $\approx 0$ , the component of **r\_fwd** that is perpendicular to the wall (shown in green) will be roughly equal to **r\_perp** (shown in purple). During our testing, the lidar data for **f\_fwd**, or **f\_perp** would sometimes be invalid for long periods of time. This would lead to the Neato stopping following the wall for those long periods of time. So, we made it so that if **r\_perp** or **r\_fwd** are invalid (zero or infinity) it would take data from a scan within a 15 degree range of **f\_fwd** or **r\_perp**. This way, even if the lidar scans are faulty, the neato would still be able to follow the wall. At the last minute there was a bug in the code and the neato began drifting into the wall. This issue happened after no prior code changes, we tried to fix the issue but we ran out of time. We are unsure if the issue will be there if run again.

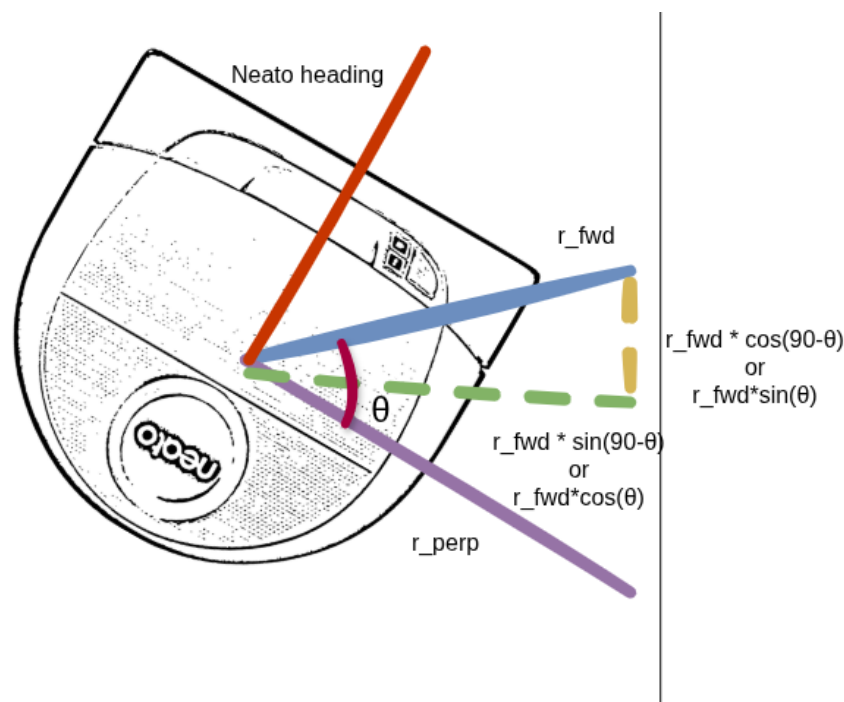


Figure 2 displays the illustration and derivation of  $\alpha = \text{math.atan2}(\text{r\_fwd} * \text{math.cos}(\text{theta}) - \text{r\_perp}, \text{r\_fwd} * \text{math.sin}(\text{theta}))$ , where the purple and blue lines are lidar scans and the green and yellow lines are components of the blue line.

## 2.2 People Following

In this mode, the neato will attempt to find a person within a certain area, follow them, and maintain a set distance from them. The neato will enter the person-following node whenever the neato detects a person. A neato detects a person by using lidar data in a given field of view. Then it bunches scans that are within **gap\_threshold = 8** degrees of each other and ranges within **cluster\_range\_jump=0.2 m** of each other into the same cluster. The Neato then filters these clusters and creates **candidates** based on whether or not each cluster seems “person-like”. A “person-like” cluster is defined by its width, with clusters in the range of **min\_person\_width=0.06** and **max\_person\_width = 0.6**. The nearest candidate is then stored as **best**, and the neato uses a proportional control loop in order to guide itself toward the center of the cluster. The selection of **best** is also influenced by the angle of the previous **best**. If one candidate is at a similar angle to the previous best candidate, then that cluster is most likely a person who just moved, and the neato will favor that cluster as **best** rather than the next closest cluster.

## 2.3 Drive in a square

After a set amount of time, the Neato will drive in a square. Moves the Neato in the square exactly once. This happens through four rotations and four movements. By default, the rotations are counterclockwise. It uses **self.count** and **self.action** to limit the number of possible actions, and **action\_time** to determine actions. When the action timer has elapsed, it will always stop temporarily before going to the next action, and it resets the action timer for the next action. When swapping actions between wall-following and driving in the square, the **self.count** and time will be reset.

## 2.4 Collision avoidance

The Neato applies collision avoidance through the lidar scanner, reading values that are spaced 45 degrees from the center of the front of the Neato, creating a 90-degree cone to read from. The threshold we set was **safety\_distance = 0.4**, which we found to work well for our code, as the Neatos have 0.4 meters of leeway. Once the Neato approaches the safety\_distance, it will begin turning at the **safety\_turn\_rate = 2**, which the units are in radians per second. Additionally, the movement of the Neato will slow down as it gets closer to that safety distance, which is governed by this equation: **msg.linear.x = max(0.0, self.forward\_vel\*(self.front\_dist / self.safety\_distance)\*\*2)**. We take the square of the ratio of the two in order to make sure negative numbers do not appear; this ratio acts as a variable factor.

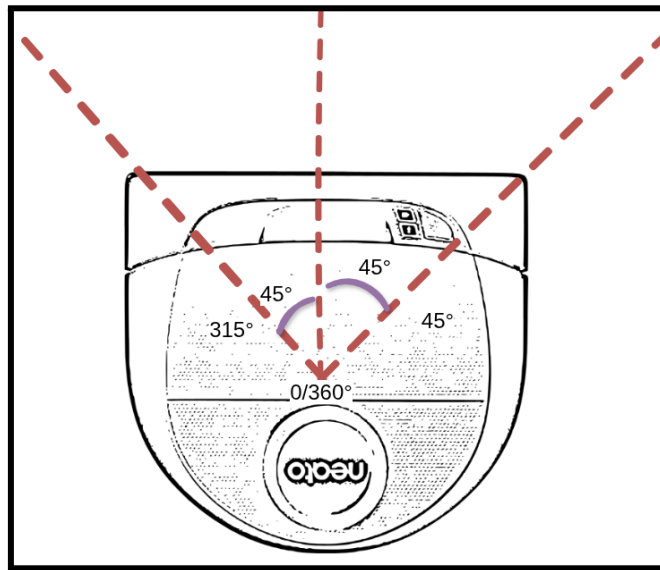


Figure 2 shows the range of the values the lidar reads from being from  $[0, 45]$  and  $[315, 360]$

### 3 Contributions, Takeaways, Future Works

Detch worked on collision avoidance, driving in a square, control node and a quick prototype of the wall-following code. Visualizations and figures used in the write-up were also created by Detch. Luke worked on wall-following using a more complex method, refactored and integrated the code into one final node to run on, as well as, creating the bag files.

Some takeaways for future robotics projects are that as behaviors got more complex, using multiple nodes and an orchestrator node would be wise. Even though we didn't plan on having a master node we ultimately decided to implement one. Another thing we learned is that the simulation and reality are entirely different realms. The gazebo simulator is useful for testing out whether or not a node's logic is correct but it doesn't guarantee it will work. For instance, our wall following program worked flawlessly in the simulator. It corrected quickly, and could follow corners. However, when we ran it on the real neato, there were several unforeseen bugs (such as the invalid readings for `r_fwd`) which took many hours to debug. If we were to do this project again we would start working on the real neatos earlier.

In the future, if we did have more time, we intended to implement other behaviors like mapping or making the Neato play a noise depending on how close it is to an object (where the frequency or pitch of the audioform would increase as it gets closer). We would also refine the person-following node. As of now, it sometimes identifies walls as people since sometimes walls get split into multiple, smaller clusters. We think this is because the distances of different scans can vary widely along a flat wall. These distances are often greater than the **cluster\_jump\_difference** and can split the wall into smaller clusters that fit the qualities of a "person-like" cluster. In the future we might add shape detection so if the cluster appears completely flat, the neato will know it is a wall. We would also like to fully debug the wall following code and fix the issue with it drifting into the wall. Additionally, we would like to explore the option of changing our proportional control loops into PID loops. Some of us have worked with PID control in the past and we think it would make the neato's movements smoother.

#### Videos:

<https://drive.google.com/file/d/1rMBP5EUhB0se3zI0MB0AYN0OieJckXz/view?usp=sharing>

[https://drive.google.com/file/d/1wNdTYjX-ahMoLfMG7qvB\\_rzA1FXLZQvb/view?usp=sharing](https://drive.google.com/file/d/1wNdTYjX-ahMoLfMG7qvB_rzA1FXLZQvb/view?usp=sharing)